CSE 391: System & Software Tools

Diving Deeper into git

Ruth Anderson
✉️ rea@cs.uw.edu
🔗 cs.uw.edu/391

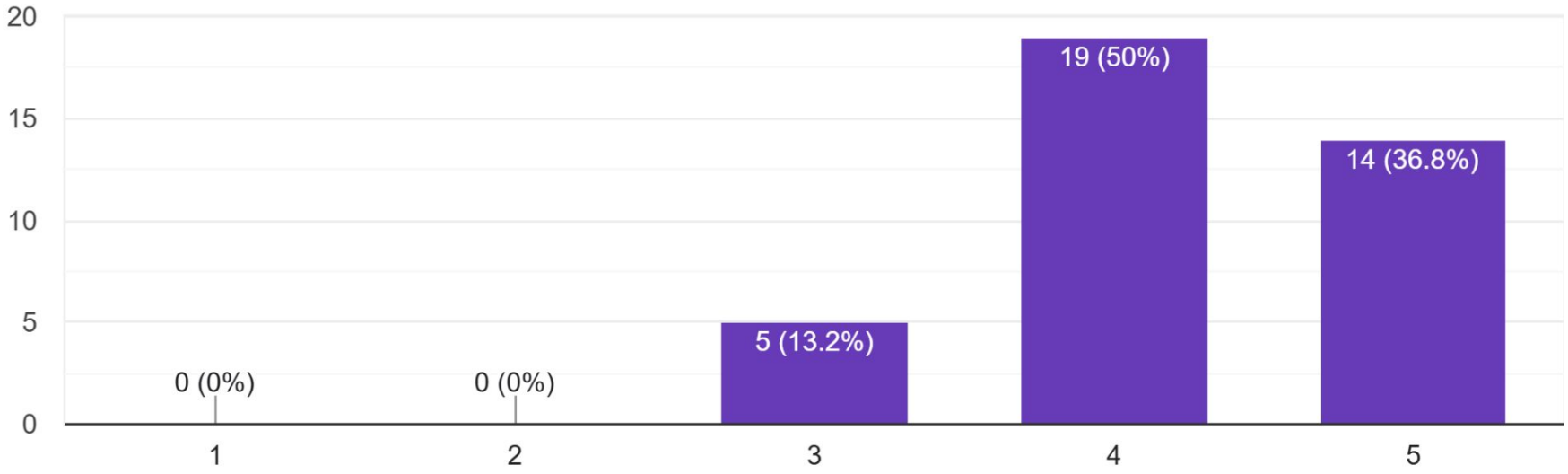
xkcd #1597

Agenda

- Midquarter Feedback
- HW4 Questions
- HW5
- Discussion Questions

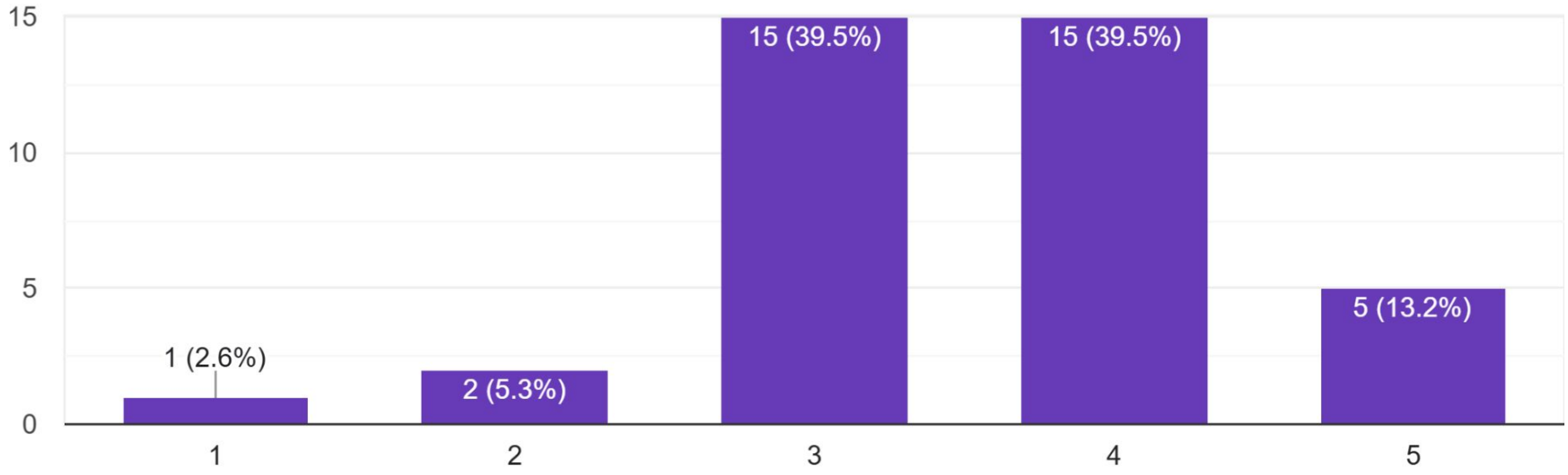
How effective are the pre-lecture videos in teaching you skills in this class?

38 responses



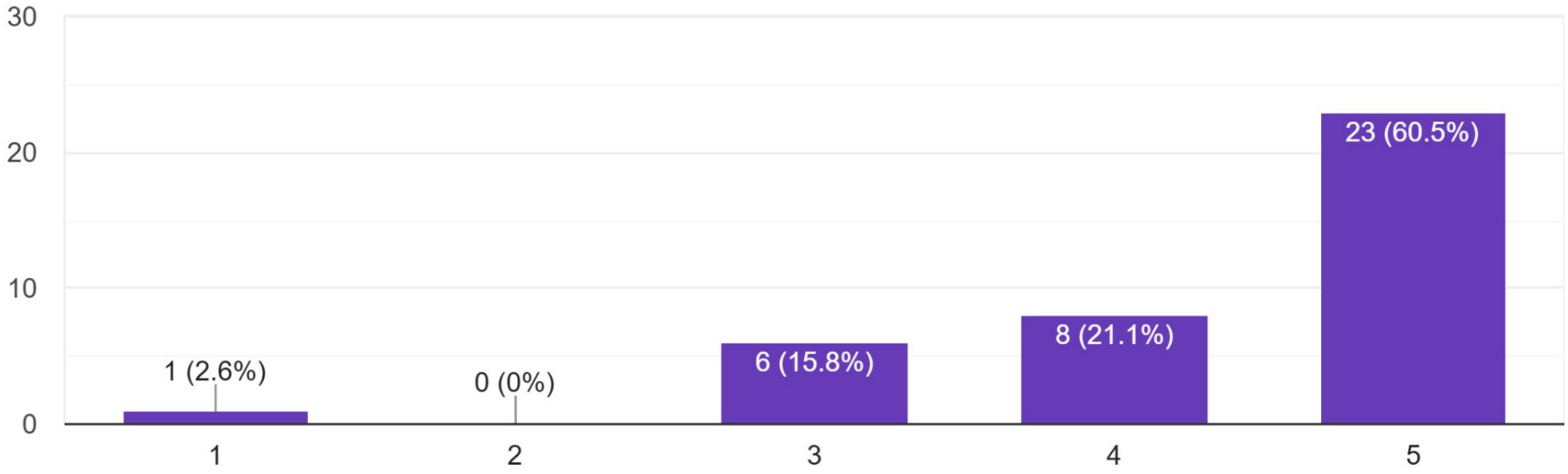
How effective are the live lectures in teaching you skills in this class?

38 responses



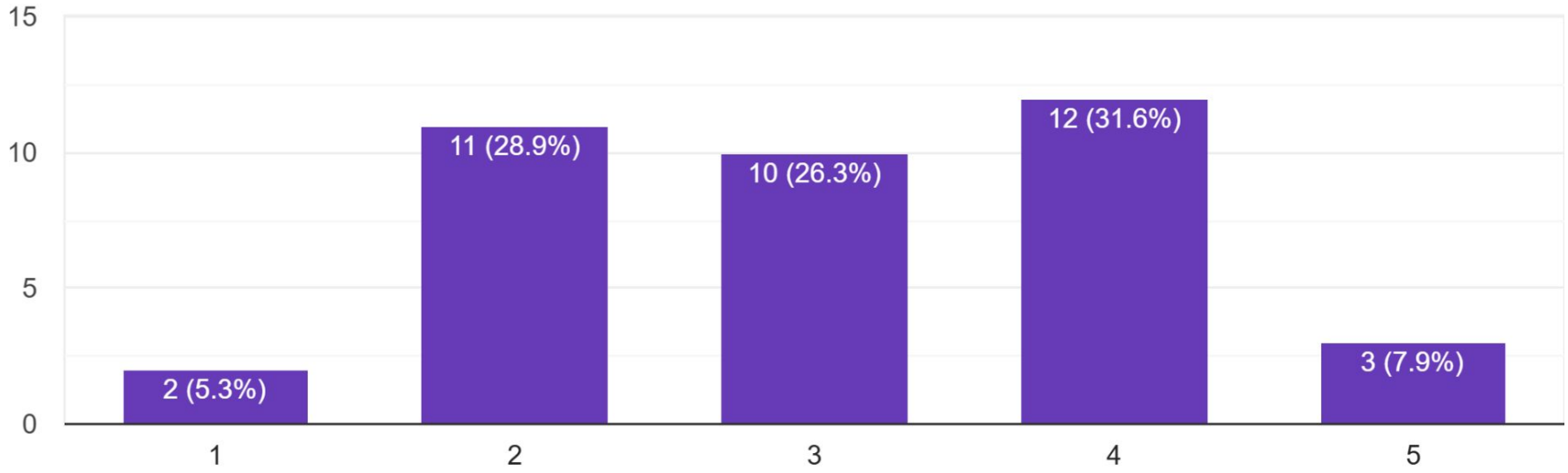
How effective are the homeworks in teaching you skills in this class?

38 responses



Do you like that this class is "flipped"?

38 responses



other thoughts

- thanks for giving feedback & engaging in this class!
- 1-credit class workload & support
  - homework is <u>not</u> supposed to take hours and hours – if you're stuck after 15-20 minutes, please reach out and ask for help!
  - happy to support via EdStem, remote or in-person office hours
  - **does require you to start somewhat early**
- have other thoughts? feel free to let me know...
  - via email, Ed, or during office hours (great since I can ask you follow-ups)
  - or via the CSE anonymous feedback tool (privacy-preserving, though I can't follow-up)

HW4 Q&A + review

other HW4 questions?

HW5 in a nutshell

The entire class will edit a website together (through GitLab)!

- you need to add “yourself” + a “product” to the site (+ 2 photos)
- you will test + add, commit, & push your changes to your own branch
- then, you will create a merge request, comment, and finally merge it
- may have some merge conflicts :-)

The “FAANG” repo

You should have access to the 26wi/FAANG repo!

New things in HW5:

- browsing the merge requests & pipelines features
- building the site from source (this is a “static site generator”)

Discussion Questions

- Start with Discussion Question 1, now - this is good practice for HW5!
- Do as many of the others as you can.

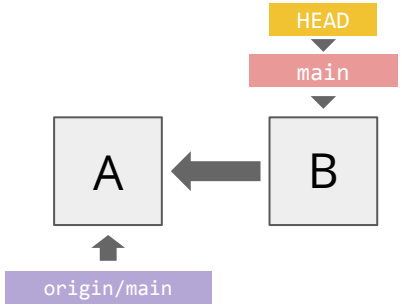
rest: pre-lecture slides (what you saw in the video)

Quick Recap - git mental model

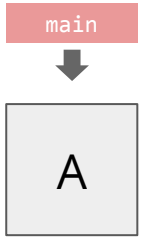
- **repository**: copy of all files (and changes)
- **commit**: a group of changes to files
- **branch**: a pointer to a commit
- **HEAD**: the branch/commit you are currently working on
- four stages of git:
  1. working directory
  2. staging area
  3. local repository
  4. remote repository

git push (1/2)

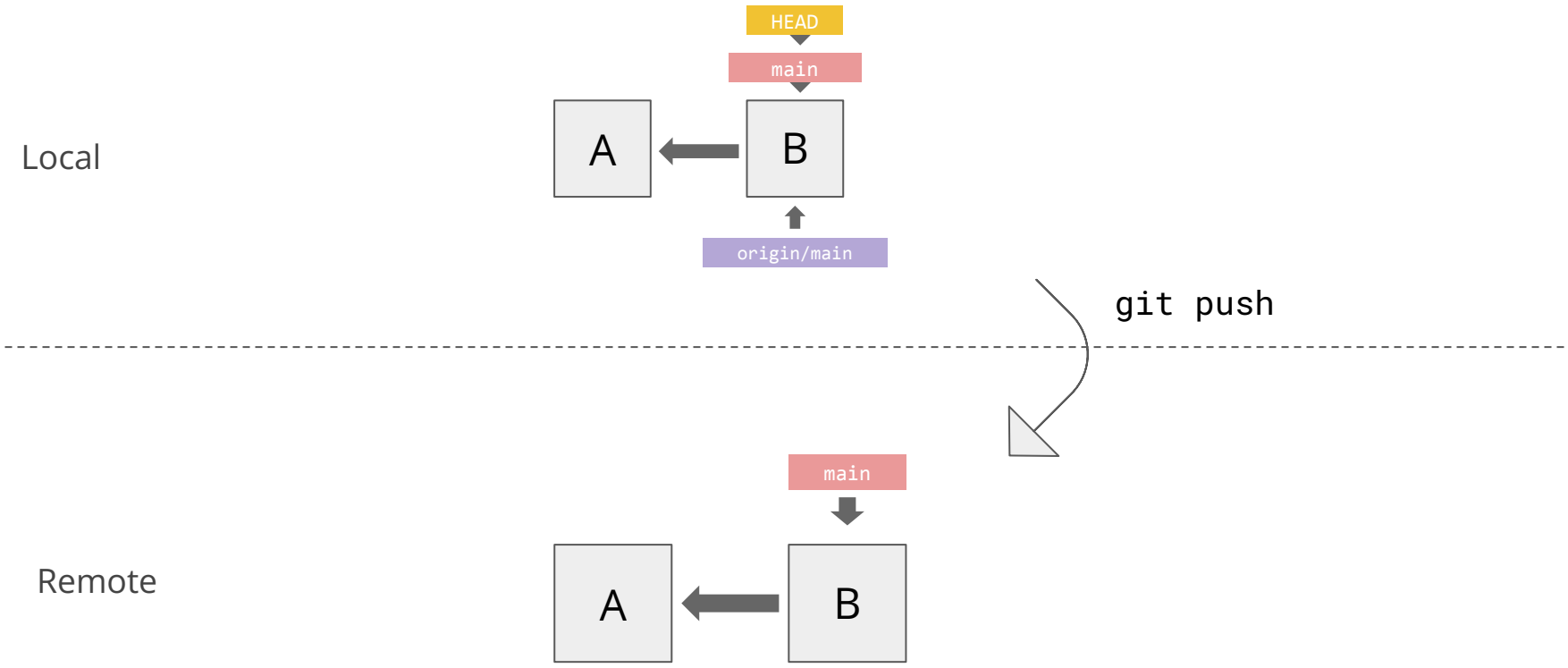
Local



Remote

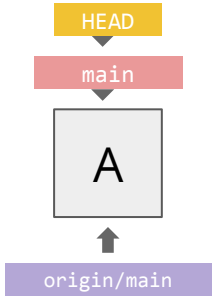


git push (2/2)

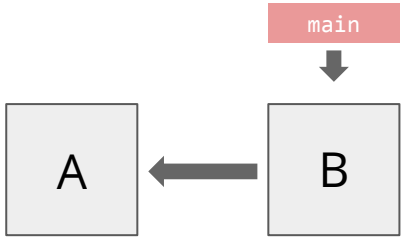


git pull (1/2)

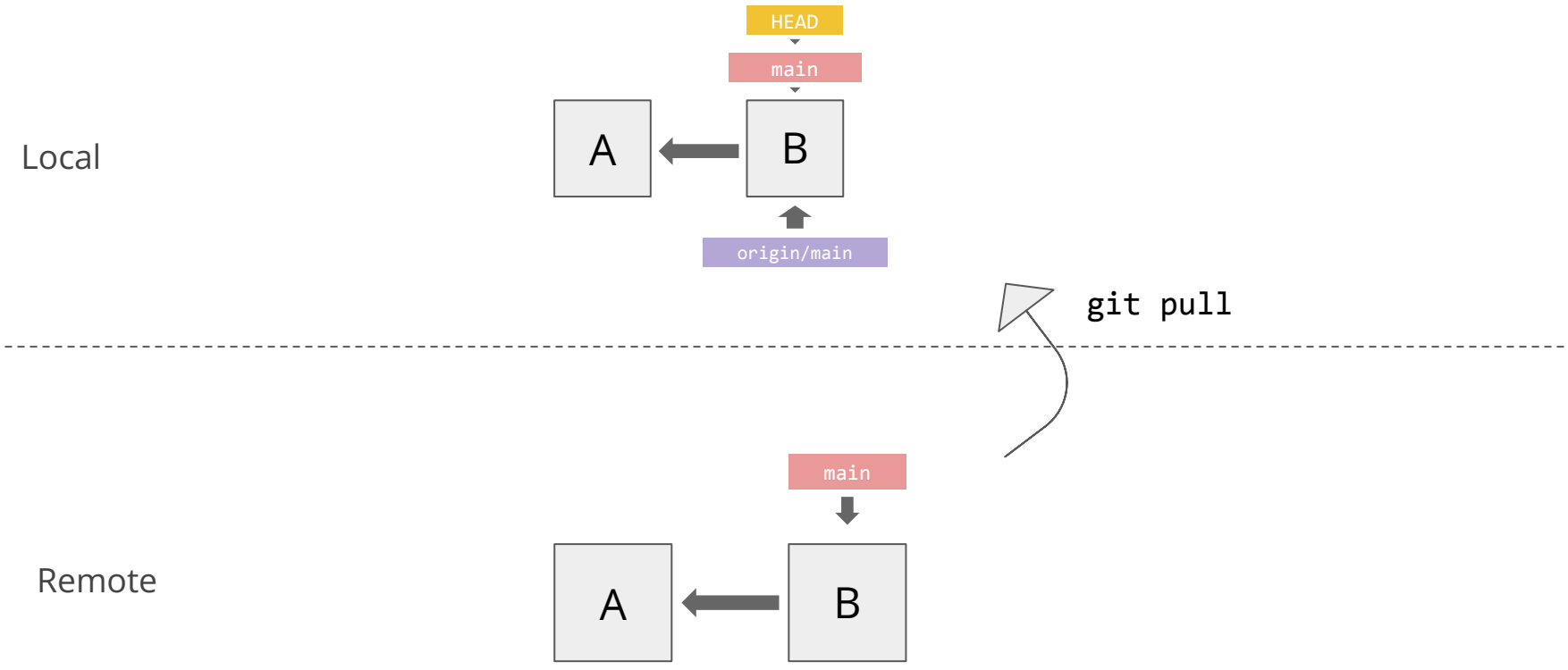
Local



Remote

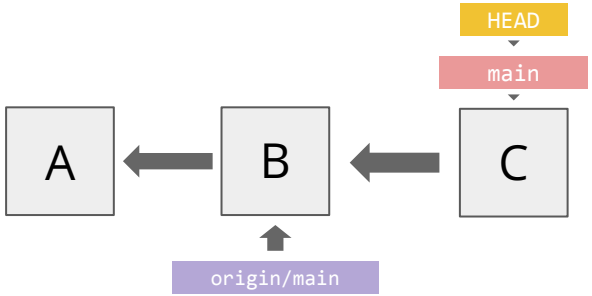


git pull (2/2)

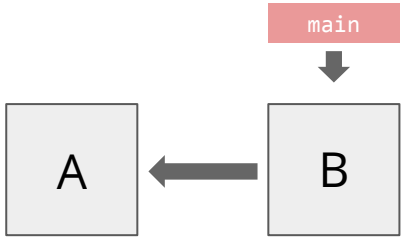


diverging commits (1/3)

Local

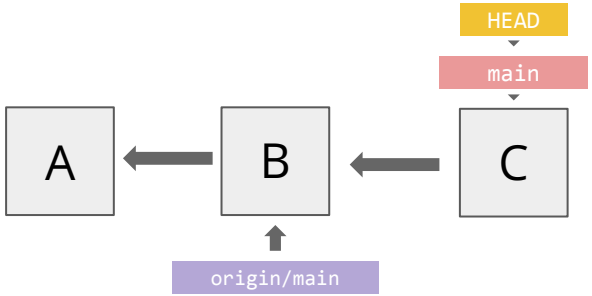


Remote

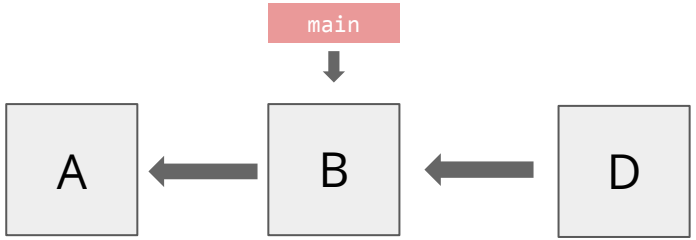


diverging commits (2/3)

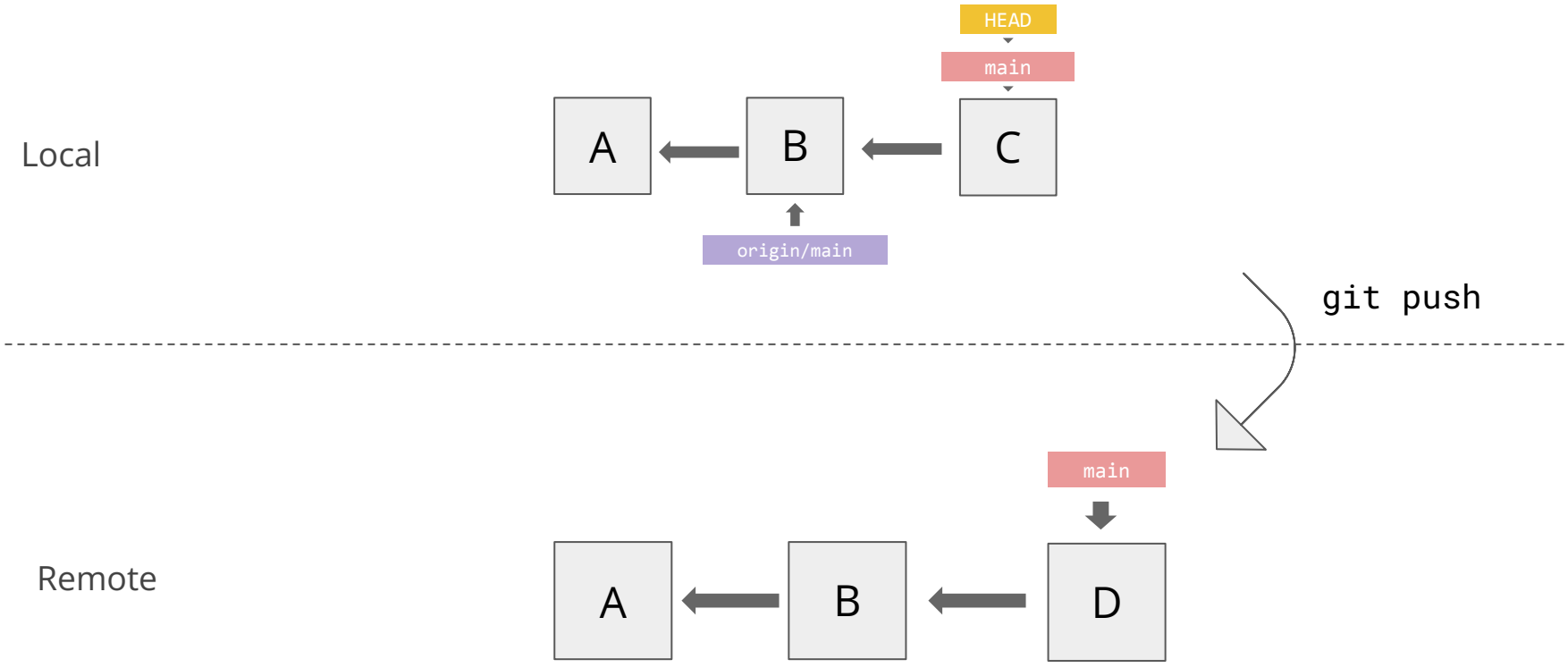
Local



Remote

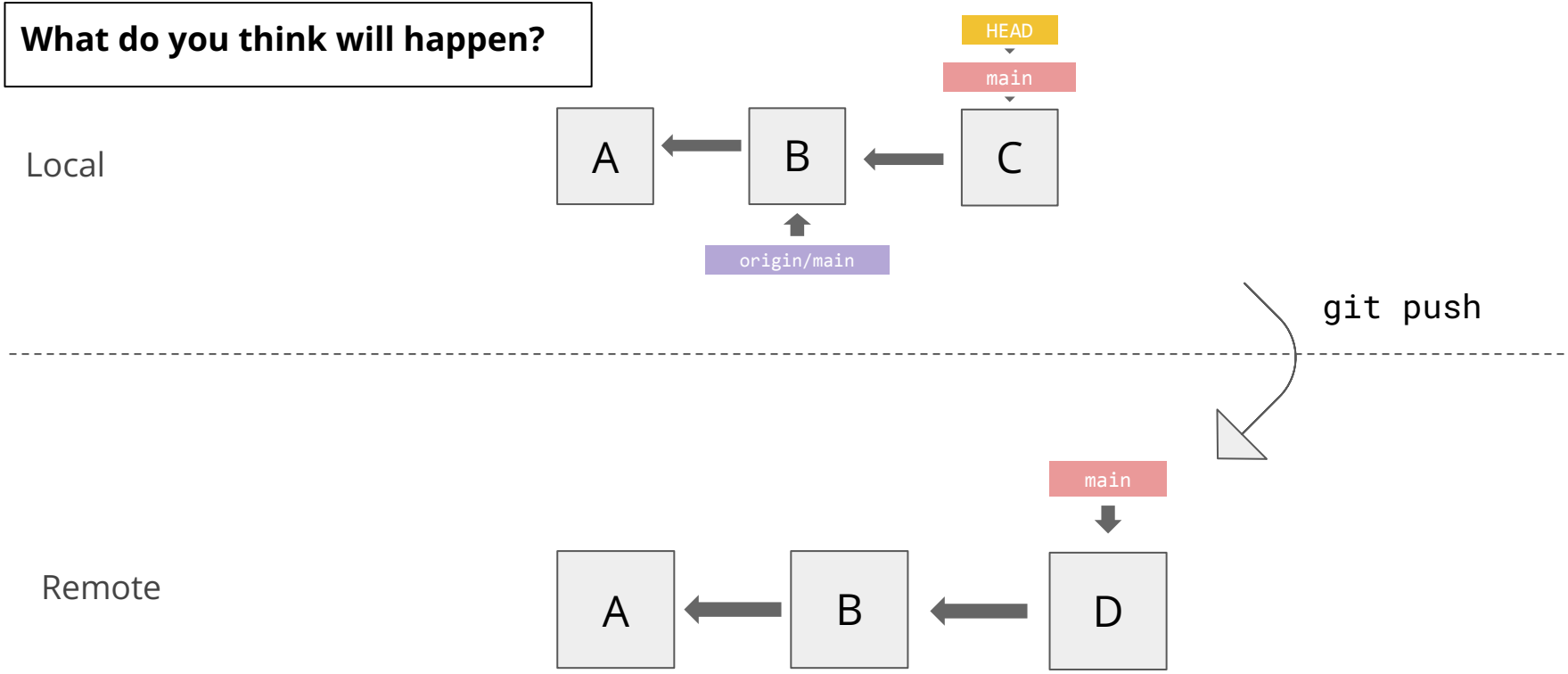


diverging commits (3/3)

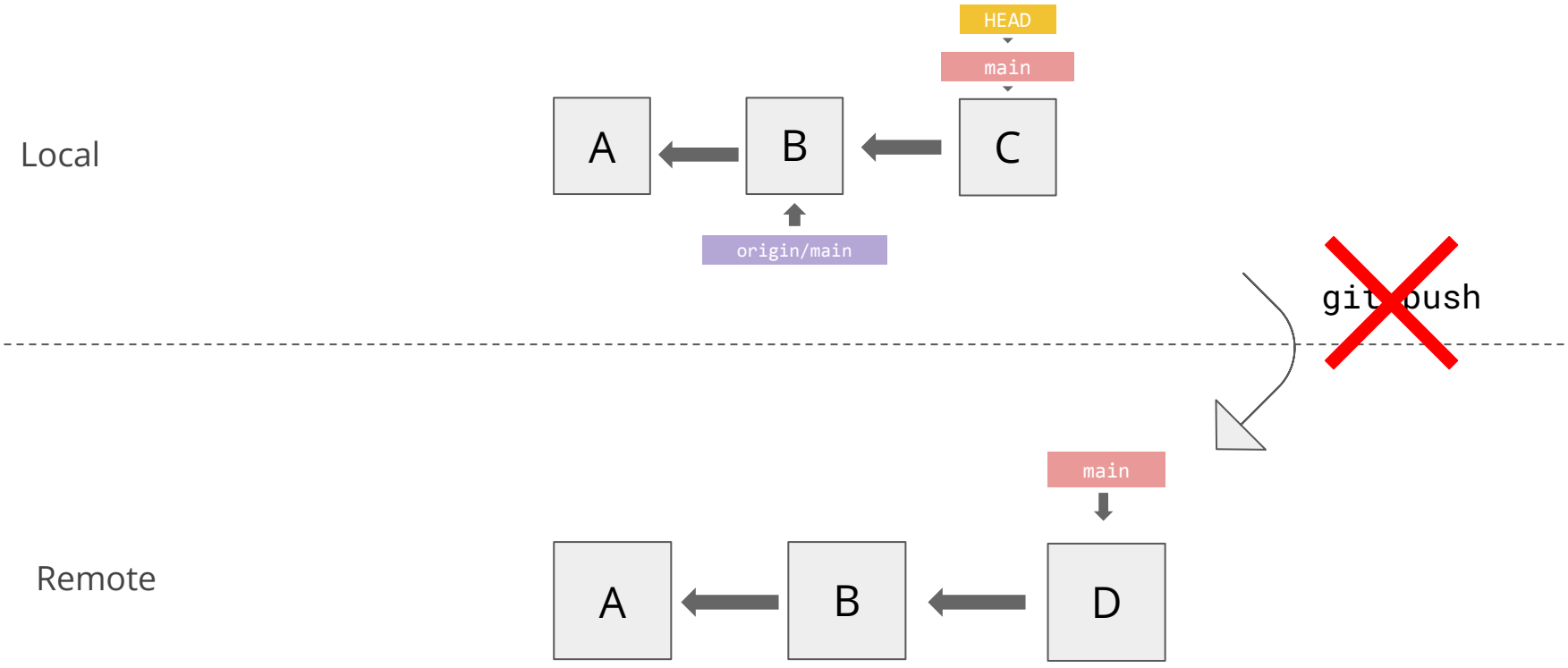


🤔 Pause and Think (Plentiful Pushes)

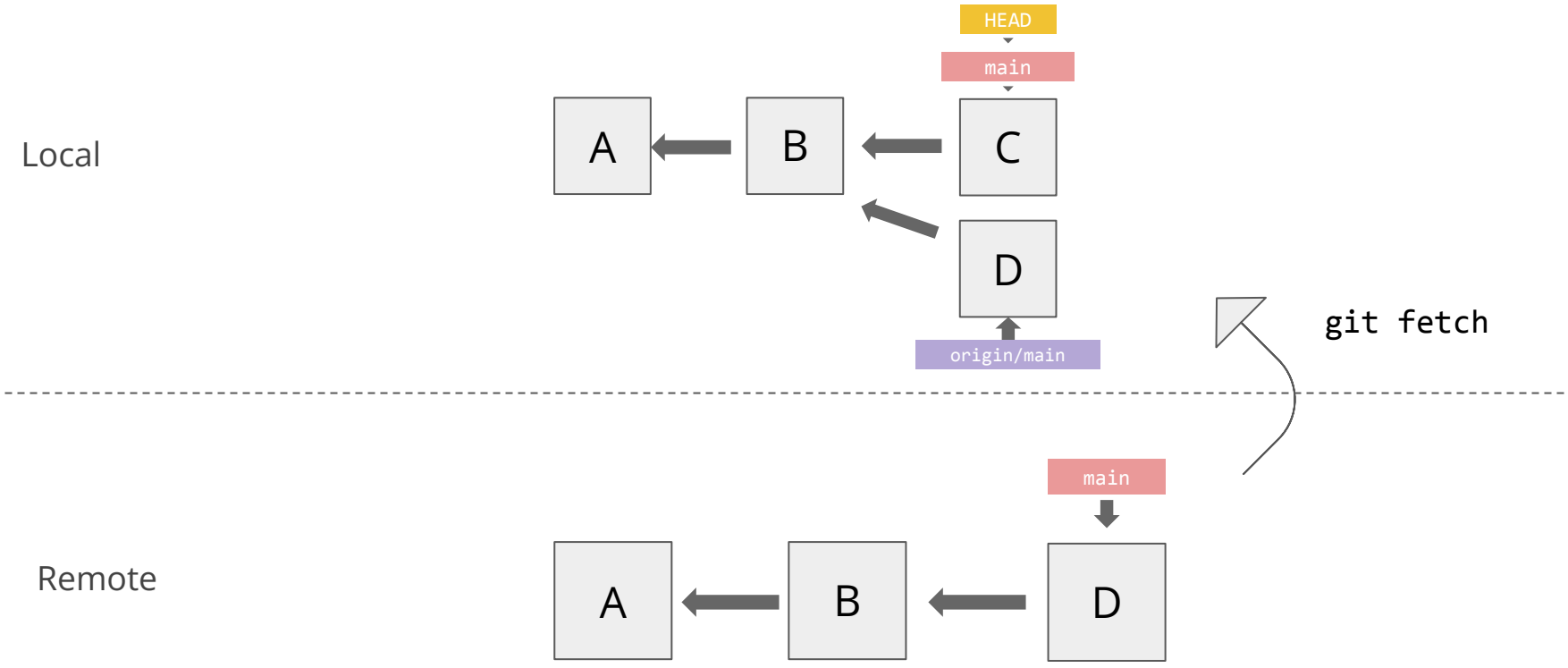
What do you think will happen?



resolving remote conflicts (1/4)

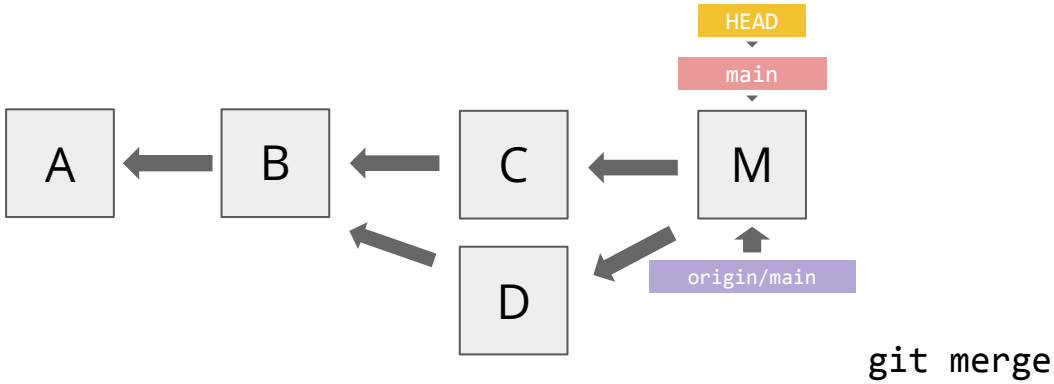


resolving remote conflicts (2/4)

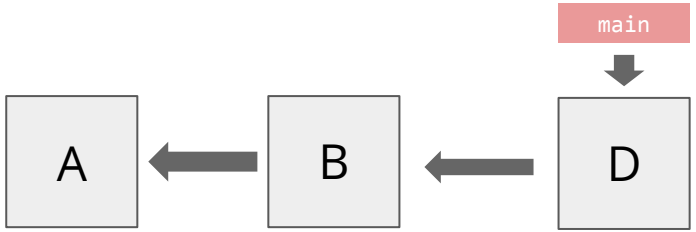


resolving remote conflicts (3/4)

Local

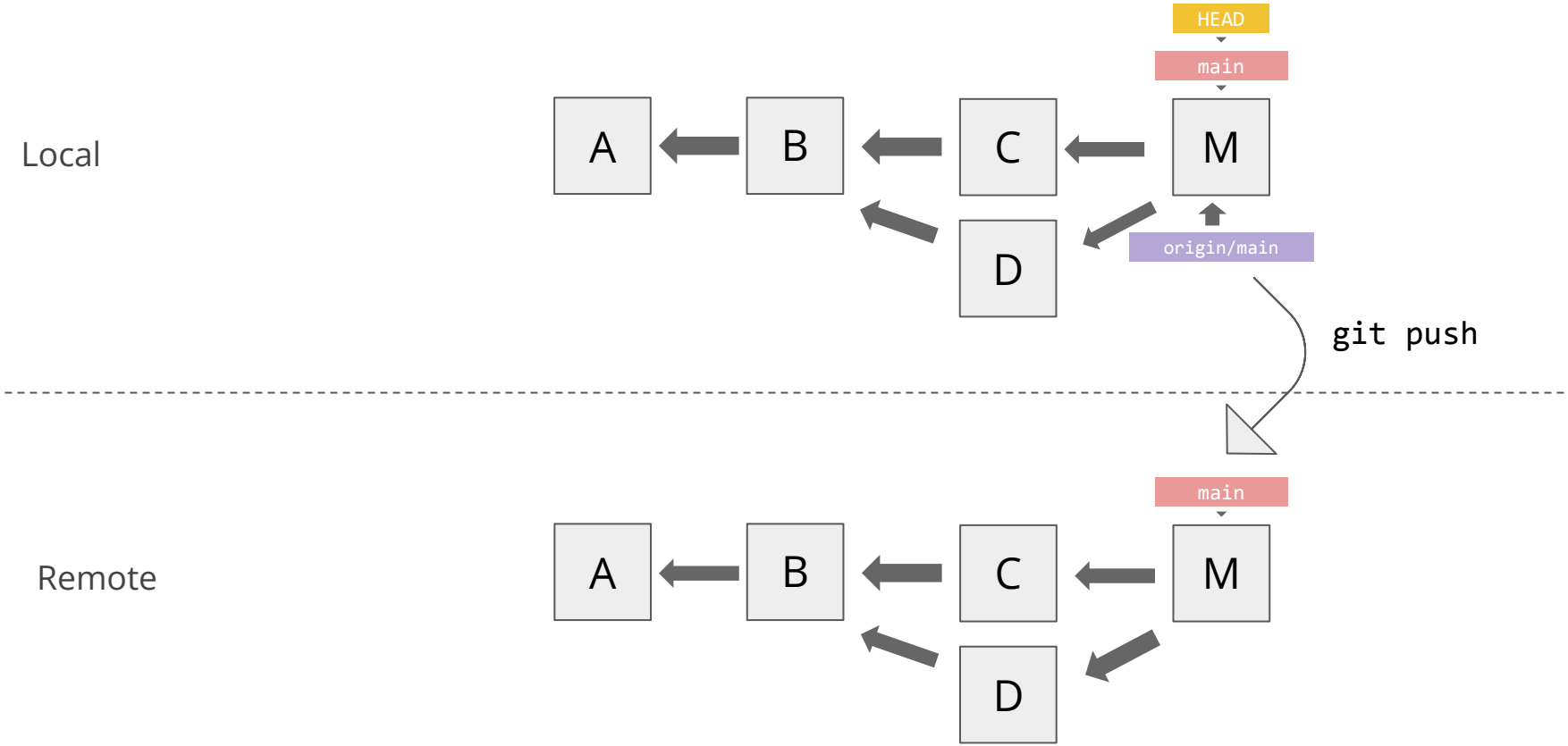


Remote



`git pull` is basically just an alias for `git fetch` + `git merge`!

resolving remote conflicts (4/4)



pull & merge requests

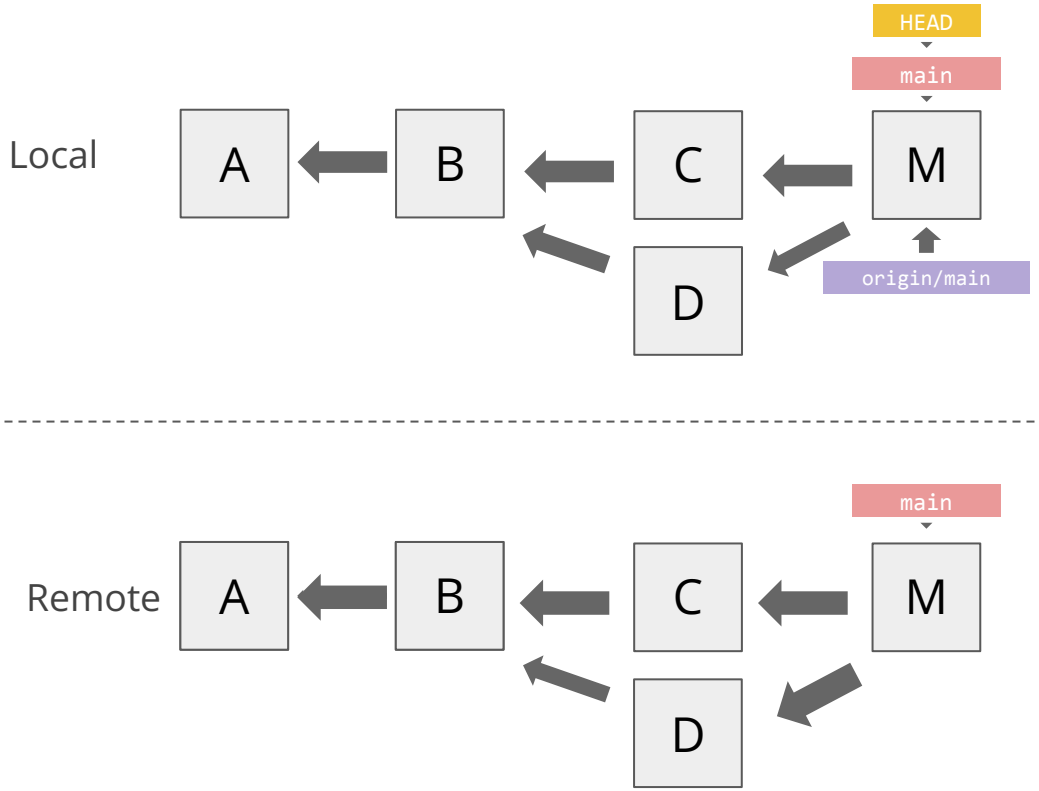
In practice, we often use tools like GitHub or GitLab to help us merge:

1. create a local branch & make some commits
2. push those commits + branch to remote (e.g. GitHub or GitLab repo)
3. open a pull request (GitHub) or merge request (GitLab)
4. collaborate with others (e.g. code review & comments), fix conflicts
5. merge into `main` (or other branch)

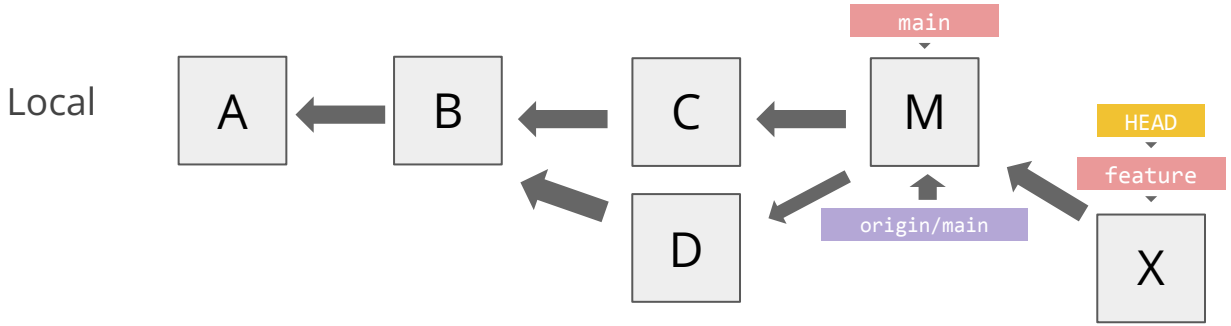
While this means we don't typically run `git merge` manually,

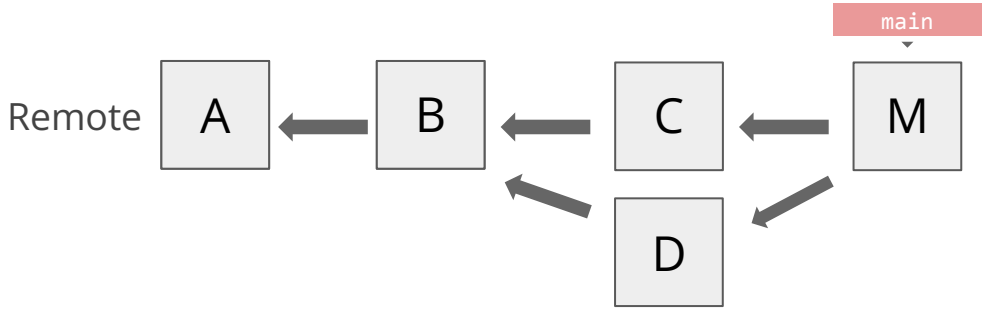
- it's important to know how it works – that's what GitHub & GitLab do!
- you may have to use it for tougher merge conflicts

remote & branches (1/6)

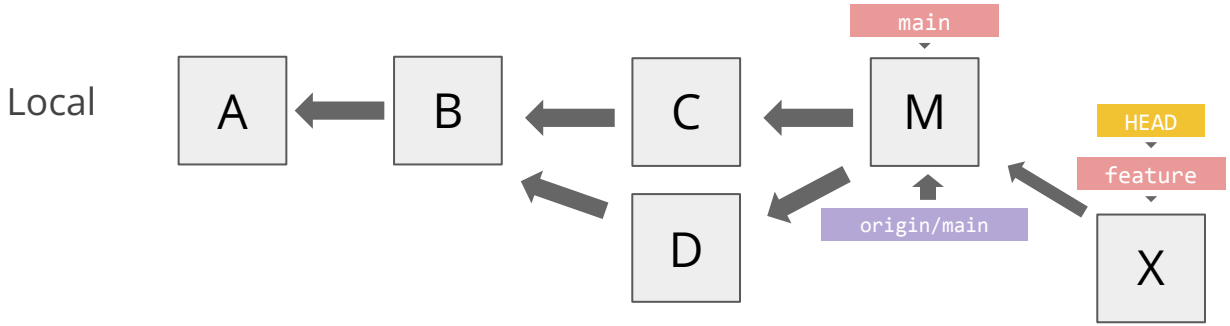


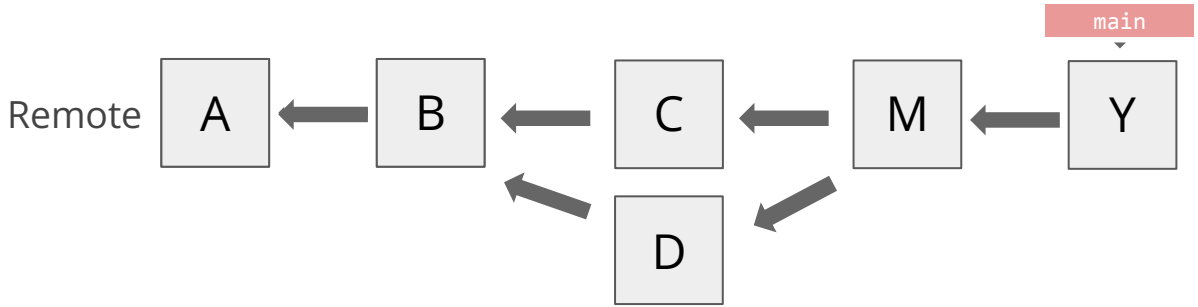
remote & branches (2/6)





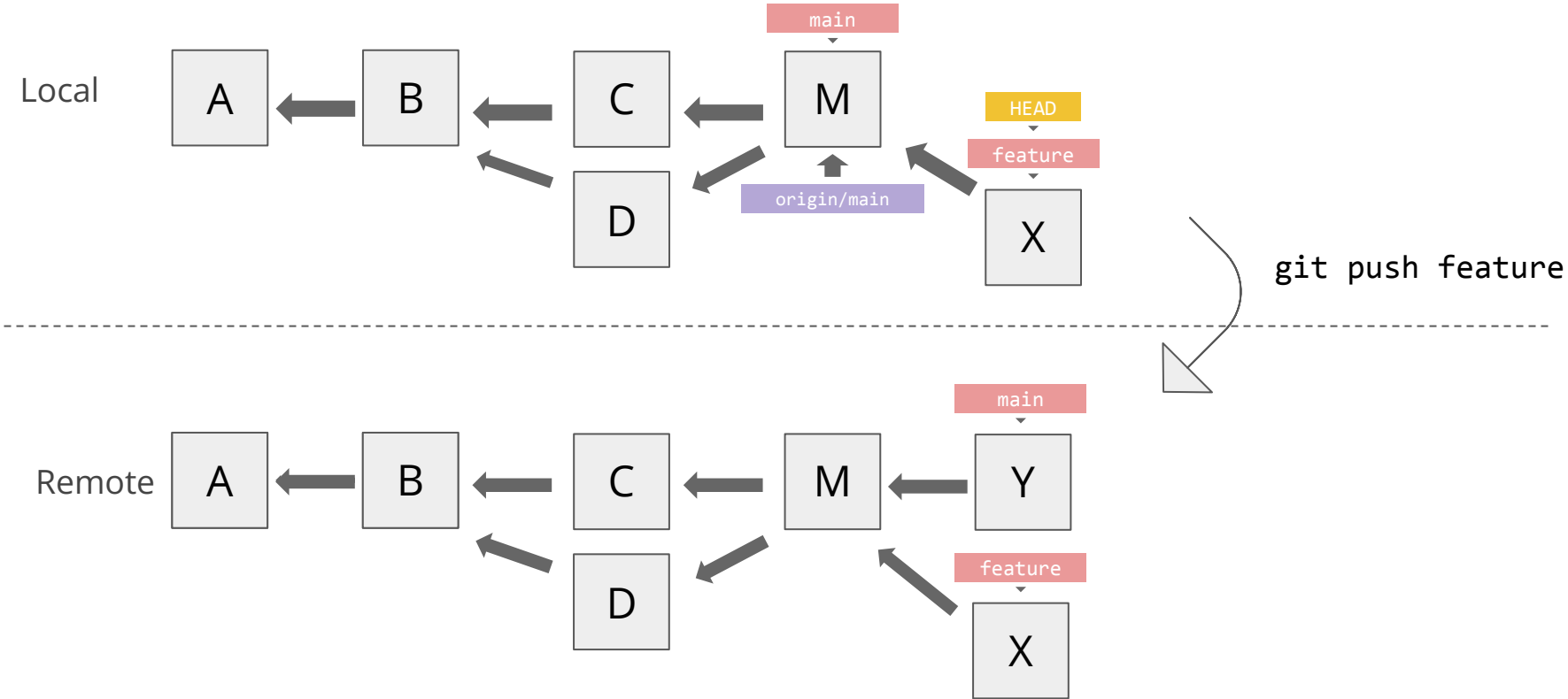
remote & branches (3/6)



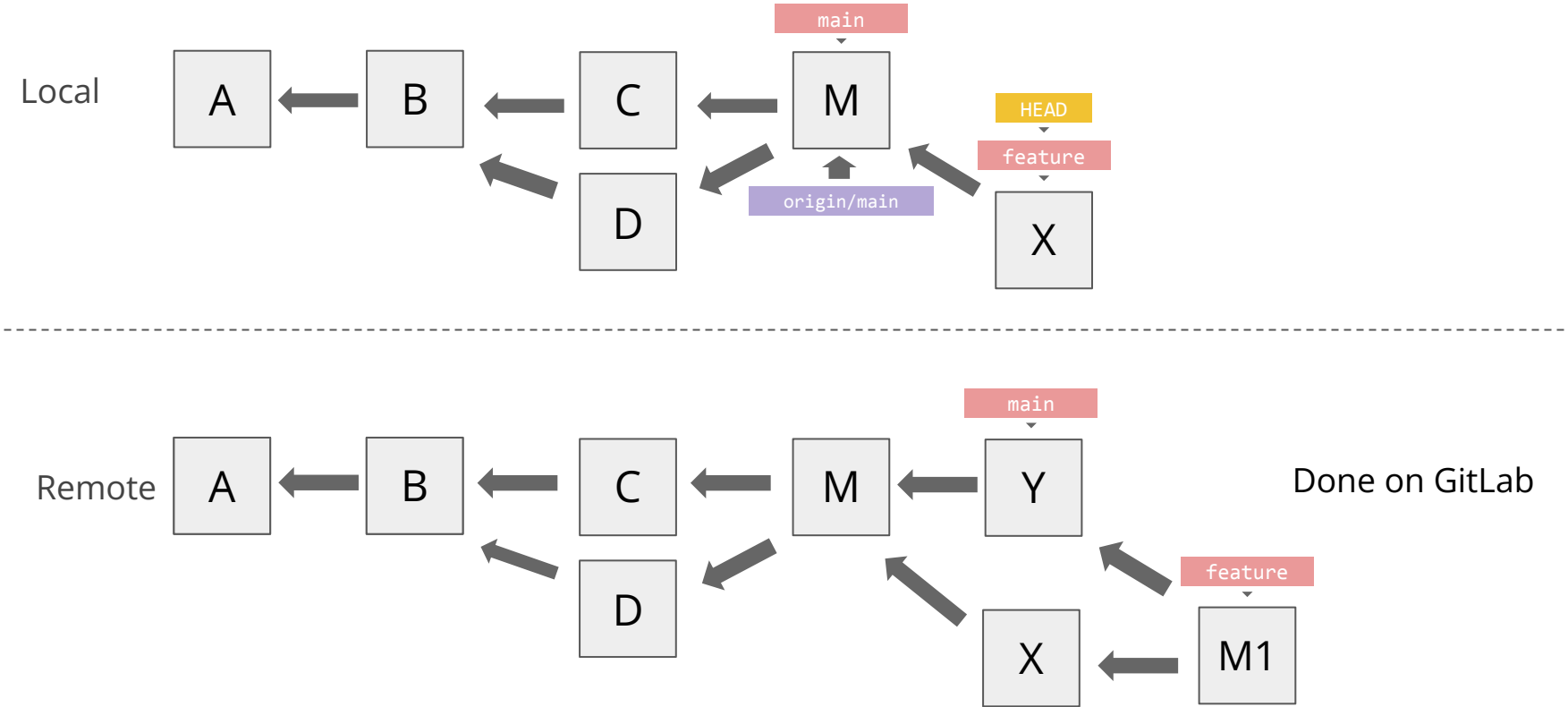


Commit happens on origin

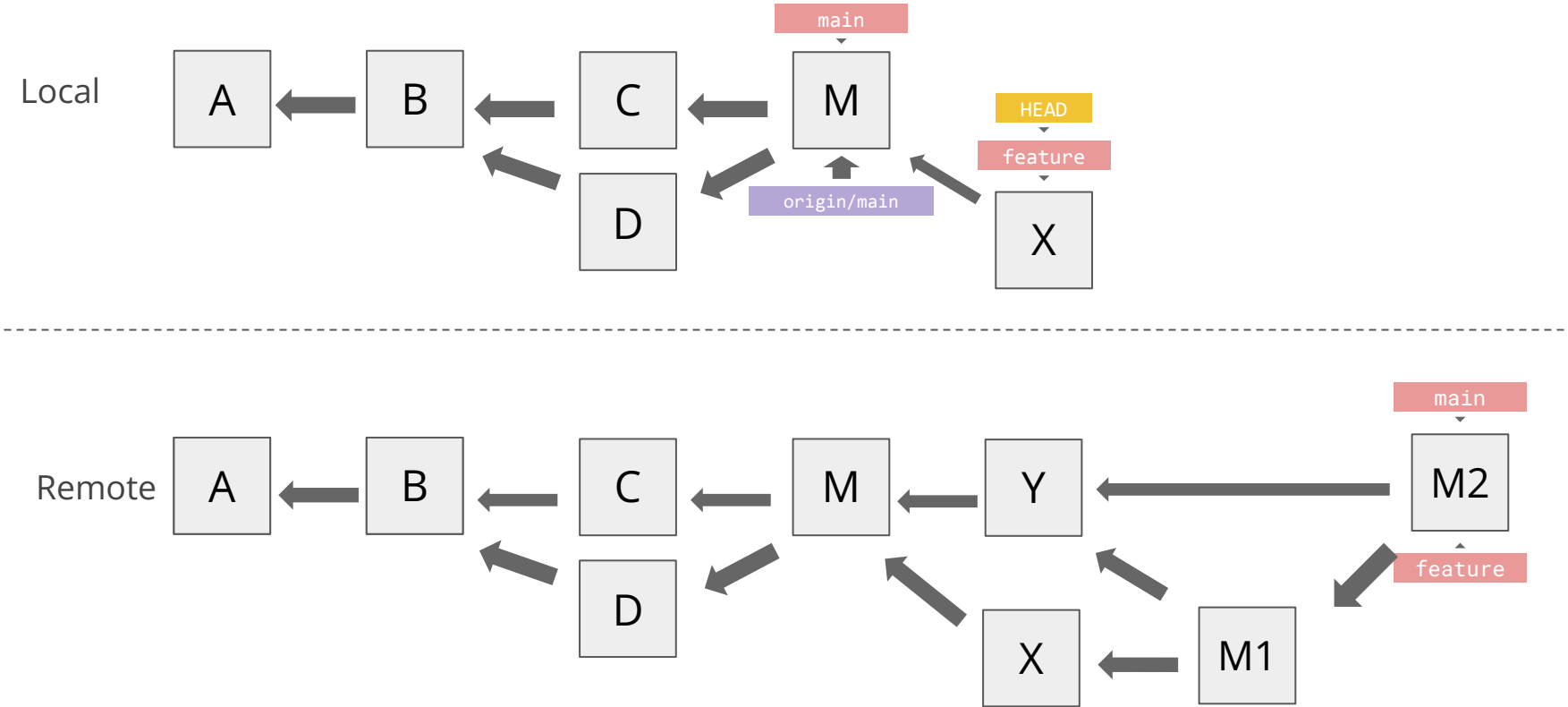
remote & branches (4/6)



remote & branches (5/6)



remote & branches (6/6)



more git: reverting

- `git revert`: adds a new commit undoing the change
  - pro: does not alter the previous git history (easy to work with others)
  - con: does not alter the previous git history (does not hide information)
- `git reset` & force-push: move HEAD to a previous commit
  - pro: alters the git history (removes sensitive information)
  - con: alters the git history (causes irreconcilable changes with collaborators)
- our advice:
  - on `main` and other shared branches: do not alter the git history
  - on your own, private branch: do what you want, but ideally avoid force-pushing

more git: merge strategies

Many different ways to merge – what we learned is called a “merge commit”.

Other options include:

- `git rebase`: “**re**apply” branch commits to a different **base** branch
- “squash” merging: rewrite history to make all changes 1 commit
- other: fast-forward, custom strategies (e.g. handling 3-way merges)

Mostly out of scope of this class, but <u>very</u> relevant in bigger projects!